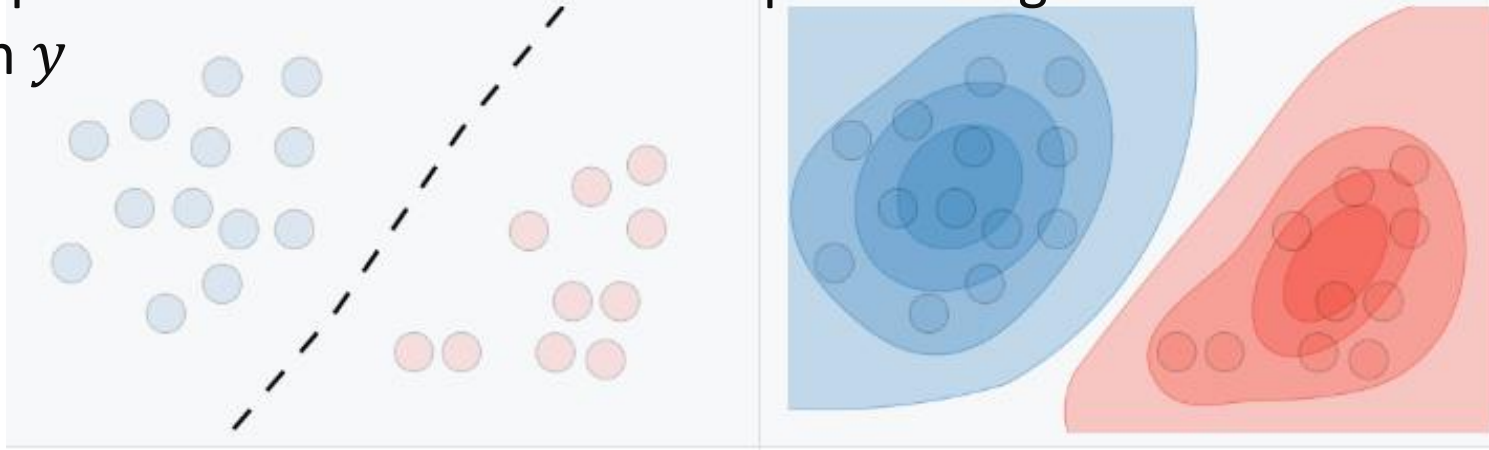


Lecture 9 Generative Model and Text Classification

Recap: Generative vs Discriminative Model

- Discriminative, $p_{\theta}(y|\mathbf{x})$:
 - Given a input data $\mathbf{x}$
 - Compute the likelihood of each class y
- Generative, $p_{\theta}(\mathbf{x}|y)$:
 - Given a label class y
 - Compute the likelihood that data point $\mathbf{x}$ is generated from y



Generative vs Discriminative

- Recall discriminative model (logistic regression)

$$P_{\theta}(y|\mathbf{x}) = \prod_{i=1}^n P_{\theta}(y^{(i)}|x^{(i)})$$

- If we only care about prediction, we don't need a model of $P(\mathbf{x})$.
- It is simpler to only model $P(y|\mathbf{x})$.
- In practice, discriminative models are often more accurate
- However, discriminative models often requires large sample sizes to learn the decision boundaries well

Generative vs Discriminative

- Generative models:

$$\hat{y} = \arg \max_y p_{\theta}(x|y)p_{\theta}(y)$$

- It requires less samples
- But need to make fewer samples strong assumptions to learn

$P(x, y)$ -> higher bias (less accurate)

- Use generative approach if we care about other tasks
 - Data imputation and handling missing data
 - Anomaly Detection
 - Generate new data samples

Maximum Likelihood Learning

- We can learn a generative model $p_{\theta}(x, y)$ by maximizing the likelihood:

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)}).$$

- Choose parameters θ such that the model P_{θ} assigns a high probability to each training example $(x^{(i)}, y^{(i)})$ in the dataset D .
- This is probability of observing the dataset $D = \{(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)\}$ if data were IID draws from P_{θ}

Example: Text Classification

- Includes a lot of applied problems:
 - Spam filtering, fraud detection, medical record classification, etc.
- Inputs x are sequences of words of an arbitrary length
- Our algorithms take as input column vectors of a fixed dimension
 - Need to preprocess

Dataset: Twenty Newsgroups

- Contains ~20,000 documents collected approximately evenly from 20 different online newsgroups
- Each newsgroup covers a different topic
 - Medicine, computer graphics, or religion.
- This dataset is often used to benchmark text classification and other types of algorithms

The 20 newsgroups text dataset

The 20 newsgroups dataset comprises around 18000 newsgroups posts on 20 topics split in two subsets: one for training (or development) and the other one for testing (or for performance evaluation). The split between the train and test set is based upon a messages posted before and after a specific date.

This module contains two loaders. The first one, :func:`sklearn.datasets.fetch\_20newsgroups`, returns a list of the raw texts that can be fed to text feature extractors such as :class:`sklearn.feature\_extraction.text.CountVectorizer` with custom parameters so as to extract feature vectors. The second one, :func:`sklearn.datasets.fetch\_20newsgroups\_vectorized`, returns ready-to-use features, i.e., it is not necessary to use a feature extractor.

****Data Set Characteristics:****

===== Classes	===== 20
Samples total	18846
Dimensionality	1
Features	text
=====	=====


```
1 # Let's examine one data point
2 print(twenty_train.data[3])
```

From: s0612596@let.rug.nl (M.M. Zwart)
Subject: catholic church poland
Organization: Faculteit der Letteren, Rijksuniversiteit Groningen, NL
Lines: 10

Hello,

I'm writing a paper on the role of the catholic church in Poland after 1989. Can anyone tell me more about this, or fill me in on recent books/articles(in english, german or french). Most important for me is the role of the church concerning the abortion-law, religious education at schools, birth-control and the relation church-state(government). Thanx,

Masja,

"M.M.Zwart"<s0612596@let.rug.nl>

Feature Representations for Text

- Each data point x in this dataset is a sequence of characters of an arbitrary length
- Might want to hand-craft features by inspecting the data:
 - Does the message contain the word “church”?
 - Does the email of the user originate outside the United States?
 - Is the organization a university?
 - ...

Feature Representations for Text

- Might want to hand-craft features by inspecting the data
- Count the number of occurrences of each word:
 - Does this message contain “church”, yes or no?
 - Does this message contain “doctor”, yes or no?
 - Does this message contain “purple”, yes or no?
- Many modern deep learning methods can directly work with sequences of characters of an arbitrary length

Bag of Words Representations

- Start by defining a vocabulary V
 - Containing all possible words that we are interested in
 - E.g., $V = \{\text{church, doctor, purple, slow, apple, ...}\}$
- A bag of words representation of a document x is a function $\phi(x) \rightarrow \{0,1\}^{|V|}$

- Outputs a feature vector of dimension V

- j -th component $\phi(x)_j = 1$ if
 - x contains the j -th word in V
 - 0 otherwise

$$\phi(x) = \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \\ \vdots \end{pmatrix} \begin{matrix} \text{church} \\ \text{doctor} \\ \text{purple} \\ \\ \text{slow} \end{matrix}$$

Sklearn

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 # vectorize the training set
4 count_vect = CountVectorizer(binary=True)
5 X_train = count_vect.fit_transform(twenty_train.data)
6 X_train.shape
```

(2257, 35788)

- Don't need a predefined dictionary
- Fit_transform: learn all raw tokens and convert the data to sparse matrix representation
- Binary: whether the document contains the word or not

Retrieve the index of $\phi(x)$ associated with each word using `count_vect.vocabulary_.get(word)`:

```
1 # The CountVectorizer class records the index j associated with each word in V
2 print('Index for the word "church": ', count_vect.vocabulary_.get(u'church'))
3 print('Index for the word "computer": ', count_vect.vocabulary_.get(u'computer'))
```

```
Index for the word "church": 8609
Index for the word "computer": 9338
```

- Our featurized dataset is in the matrix `X_train`.
- Retrieve the 0-1 value computed for each word using above indices

```

1 # We can examine if any of these words are present in our p
2 print(twenty_train.data[3])
3
4 # let's see if it contains these two words?
5 print('---'*20)
6 print('Value at the index for the word "church": ',
7       X_train[3, count_vect.vocabulary_.get(u'church')])
8 print('Value at the index for the word "computer": ',
9       X_train[3, count_vect.vocabulary_.get(u'computer')])
10 print('Value at the index for the word "doctor": ',
11       X_train[3, count_vect.vocabulary_.get(u'doctor')])
12 print('Value at the index for the word "important": ',
13       X_train[3, count_vect.vocabulary_.get(u'important')])

```

I'm writing a paper on the role of the catholic church in Poland
 Can anyone tell me more about this, or fill me in on recent book
 in english, german or french). Most important for me is the role
 church concerning the abortion-law, religious education at schoo
 birth-control and the relation church-state(government). Thanx,

Masja,

"M.M.Zwart"<s0612596@let.rug.nl>

```

Value at the index for the word "church": 1
Value at the index for the word "computer": 0
Value at the index for the word "doctor": 0
Value at the index for the word "important": 1

```

Practical Considerations

- Feature $\phi(x)_j$ for the j -th word
 - The count of occurrences of word j
- Preprocess raw text:
 - Stemming: keep the root of the word
 - E.g., “slowly” and “slowness” both map to “slow”
- Filtering for common stopwords
 - E.g., “the”, “a”, “and”
- Exclude rare words

Classification Using Bag of Words Features

- Logistic regression

Slide 17

```
1 from sklearn.linear_model import LogisticRegression
2
3 # Create an instance of Softmax and fit the data.
4 logreg = LogisticRegression(C=1e5, multi_class='multinomial', verbose=True)
5 logreg.fit(X_train, twenty_train.target)
```

```
[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.
[Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 1.0s finished
```

```
LogisticRegression(C=100000.0, multi_class='multinomial', verbose=True)
```

- Use this model for predicting on new inputs

```
1 docs_new = ['God is love', 'OpenGL on the GPU is fast']
2
3 X_new = count_vect.transform(docs_new)
4 predicted = logreg.predict(X_new)
5
6 for doc, category in zip(docs_new, predicted):
7     print('%r => %s' % (doc, twenty_train.target_names[category]))
```

```
'God is love' => soc.religion.christian
'OpenGL on the GPU is fast' => comp.graphics
```

Summary of Text Classification

- Normally requires specifying features over the raw data
- Bag of words: commonly used representation
 - Features are occurrences/counts of words
- Once featurized, any off-the-shelf supervised learning algorithm can be applied
 - Some work better than others
- Next, try generative model approach

A Generative Model for Text Classification

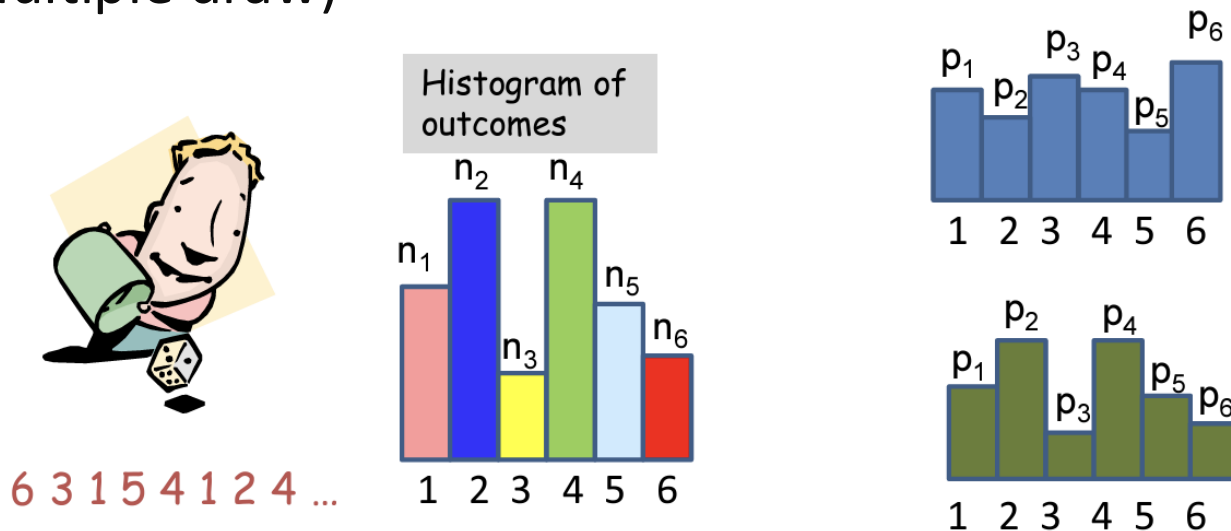
- Fit two models on a labeled corpus:

$$P_{\theta}(\mathbf{x}|y = 0) \text{ and } P_{\theta}(\mathbf{x}|y = 1)$$

- Each model $P_{\theta}(\mathbf{x}|y = k)$ scores $\mathbf{x}$ based on how much it looks like class k .
 - The documents $\mathbf{x}$ are in bag-of-words representation
 - How do we choose $P_{\theta}(\mathbf{x}|y = k)$?

Categorical Distribution

- A categorical distribution with parameter θ is a probability over K discrete outcomes
 - $a \in \{1, 2, \dots, K\}$
$$P_{\theta}(a = j) = \theta_j$$
- $K = 2$: Bernoulli distribution
- Special case of multinomial distribution (single draw vs multiple draw)



Generative Model: First Attempt

- Finite number of $\mathbf{x}$'s: binary vectors of dimension d
- $P_{\theta}(\mathbf{x}|y = k)$: categorical distribution
- Assigns a probability to each possible state of $\mathbf{x}$

$$P(\mathbf{x}|y = k) = P_k \begin{pmatrix} 0 & \text{church} \\ 1 & \text{doctor} \\ 0 & \text{fervently} \\ \vdots & \vdots \\ 0 & \text{purple} \end{pmatrix} = \theta_{\mathbf{x}k} = 0.0012$$

- $\theta_{\mathbf{x}k}$: probability of $\mathbf{x}$ under class k .
- Specify $\theta_{\mathbf{x}k}$ for all $\mathbf{x}, k$

High Dimensionality

- How many parameters does a categorical model $P_{\theta}(\mathbf{x}|y = k)$ have?
 - Say the dimension of $\mathbf{x}$ is 10000
 - $\mathbf{x}$ can take 2^{10000} values
 - Need to specify 2^{d-1} parameters for the categorical distribution
- For comparison, num of atoms in the universe $\approx 10^{82}$

Naïve Bayes Assumption

$$\hat{y} = \arg \max_y p(\mathbf{x}|y)p(y)$$

- Probability chain rule

$$\begin{aligned} p(\mathbf{x}|y) &= p(x_1, \dots, x_d|y) \\ &= p(x_1|y)p(x_2|x_1, y) \dots p(x_d|x_1, \dots, x_{d-1}, y) \end{aligned}$$

- Given a label, features are conditionally independent of each other

$$p(\mathbf{x}|y) = \prod_{i=1}^d p(x_i | y)$$

Naïve Bayes Assumption

$$p(\mathbf{x}|y) = \prod_{i=1}^d p(x_i | y)$$

- Naïve Bayes is a general technique that can be used with any d-dimensional $\mathbf{x}$ to construct tractable models $p_{\theta}(\mathbf{x}|y)$

Naïve Bayes Example

- Bernoulli model with parameter $\psi_{jk} \in [0,1]$
 - Model the probability that a document class k contains word j
 - $P_{\theta}(x_j = 1 | y = k) = \psi_{jk}$
- $P_{\theta}(x | y = k)$: product of the occurrence probabilities of each words in x in class k

$$P_{\theta}(x | y = k) = \prod_{j=1}^d P_{\theta}(x_j | y = k)$$

- How many parameters?
 - $\sim Kd$

Are the following statements on naive bayes correct?



Naïve Bayes is an assumption that we impose to reduce the number of parameters to learn the conditional distribution $P(X|Y)$

0

Naïve Bayes assumes words are uncorrected when conditioned on the label class. This assumption often holds in practice.

0

The probabilities estimated by Naïve Bayes can be overconfident or underconfident

0

Naïve Bayes often has good performance in practice despite its strong assumptions

0

Defining Prior Distributions $P(y)$

- Encodes prior belief about y before seeing
- Number of classes K is usually small, use categorical distribution with parameters $\boldsymbol{\phi} = (\phi_1, \dots, \phi_K)$
- Learn $\boldsymbol{\phi}$ from data: $P_{\theta}(y = k) = \phi_K$

Bernoulli Naïve Bayes model $P_{\theta}(x, y)$

- Binary x data, e.g., binary bag-of-words documents
- Parameter $\theta = (\phi_1, \dots, \phi_K, \psi_{11}, \dots, \psi_{dK})$:
 - prior parameters encoding $P(Y)$: $\boldsymbol{\phi} = (\phi_1, \dots, \phi_K)$
 - K sets of per-class parameters encoding $P(X|Y)$: $\psi_k = (\psi_{1k}, \dots, \psi_{dk})$
 - $K(d+1)$ parameters
- Model

$$P_{\theta}(y) = \text{Categorical}(\phi_1, \phi_2, \dots, \phi_K)$$

$$P_{\theta}(x_j = 1|y = k) = \text{Bernoulli}(\psi_{jk})$$

$$P_{\theta}(x|y = k) = \prod_{j=1}^d P_{\theta}(x_j|y = k)$$

Learning A Bernoulli Naïve Bayes Model

- Given a dataset $D = \{(x^{(i)}, y^{(i)}) \mid i = 1, 2, \dots, n\}$, optimize log likelihood

$$\begin{aligned}\ell &= \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)}) = \sum_{i=1}^n \log P_{\theta}(x^{(i)} | y^{(i)}) P_{\theta}(y^{(i)}) \\&= \sum_{i=1}^n \sum_{j=1}^d \log P_{\theta}(x_j^{(i)} | y^{(i)}) + \sum_{i=1}^n \log P_{\theta}(y^{(i)}) \\&= \underbrace{\sum_{k=1}^K \sum_{j=1}^d \sum_{i: y^{(i)}=k} \log P(x_j^{(i)} | y^{(i)}; \psi_{jk})}_{\text{all the terms that involve } \psi_{jk}} + \underbrace{\sum_{i=1}^n \log P(y^{(i)}; \phi)}_{\text{all the terms that involve } \phi} .\end{aligned}$$

Learning A Bernoulli Naïve Bayes Model

- Each ψ_{jk} only appears in

$$\max_{\psi_{jk}} \ell(\theta) = \max_{\psi_{jk}} \sum_{i: y^{(i)}=k} \log P(x_j^{(i)} | y^{(i)}; \psi_{jk}).$$

- Optimization over ψ_{jk} can be carried out independently of other parameters by just looking at these terms

Learning A Bernoulli Naïve Bayes Model

- Recall from the MLE lecture:
 - The maximum likelihood of Bernoulli distribution is the empirical count
 - ψ_{jk} : the proportion of documents in class k contains word j

Learning A Bernoulli Naïve Bayes Model

- Optimizing $\boldsymbol{\phi} = (\phi_1, \dots, \phi_K)$ only involves

$$\max_{\vec{\phi}} \sum_{i=1}^n \log P_{\theta}(x^{(i)}, y^{(i)}; \theta) = \max_{\vec{\phi}} \sum_{i=1}^n \log P_{\theta}(y^{(i)}; \phi).$$

- The optimal ϕ_K is the proportion of data points with class k (n_k) in the training set

$$\frac{\phi_k}{\sum_l \phi_l} = \frac{n_k}{n}$$

Querying the Model

$$\arg \max_y P_{\theta}(y|x) = \arg \max_y P_{\theta}(x|y)P_{\theta}(y).$$

- Estimate the probability of new data point x under each class k
 - $P_{\theta}(x|y = k) P_{\theta}(y = k)$
 - Choose the class that best explains the data

Example: Text Classification

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 # vectorize the training set
4 count_vect = CountVectorizer(binary=True, max_features=1000)
5 y_train = twenty_train.target
6 X_train = count_vect.fit_transform(twenty_train.data).toarray()
7 X_train.shape
```

(2257, 1000)

- Top 1000 frequent words
- We will learn a good set of parameters for a Bernoulli Naive Bayes model
- We will compare the outputs to the true predictions.

Compute the maximum likelihood model parameters

```
1
2 n = X_train.shape[0] # size of the dataset
3 d = X_train.shape[1] # number of features in our dataset
4 K = 4 # number of classes
5
6 # these are the shapes of the parameters
7 psis = np.zeros([K,d])
8 phis = np.zeros([K])
9
10 # we now compute the parameters
11 for k in range(K):
12     X_k = X_train[y_train == k]
13     psis[k] = np.mean(X_k, axis=0)
14     phis[k] = X_k.shape[0] / float(n)
15
16 # print out the class proportions
17 print(phis)
```

```
[0.21267169 0.25875055 0.26318121 0.26539654]
```

Compute Predictions Using Bayes' Rule

```
1  # we can implement this in numpy
2  def nb_predictions(x, psis, phis):
3      """This returns class assignments and scores under the NB
4      model. We compute  $\arg\max_y p(y|x)$  as  $\arg\max_y p(x|y)p(y)$ 
5      """
6      # adjust shapes
7      n, d = x.shape
8      x = np.reshape(x, (1, n, d))
9      psis = np.reshape(psis, (K, 1, d))
10
11     # clip probabilities to avoid log(0)
12     psis = psis.clip(1e-14, 1-1e-14)
13
14     # compute log-probabilities
15     logpy = np.log(phis).reshape([K,1])
16     logpxy = x * np.log(psis) + (1-x) * np.log(1-psis)
17     logpyx = logpxy.sum(axis=2) + logpy
18
19     return logpyx.argmax(axis=0).flatten(), logpyx.reshape([K,n])
20
21 idx, logpyx = nb_predictions(X_train, psis, phis)
22 print(idx[:10])
```

```
[1 1 3 0 3 3 3 2 2 2]
```

Measuring Accuracy

```
1 (idx==y_train).mean()
```

0.8692955250332299

```
1 docs_new = ['OpenGL on the GPU is fast']
2
3 X_new = count_vect.transform(docs_new).toarray()
4 predicted, logpyx_new = nb_predictions(X_new, psis, phis)
5
6 for doc, category in zip(docs_new, predicted):
7     print('%r => %s' % (doc, twenty_train.target_names[category]))
```

'OpenGL on the GPU is fast' => comp.graphics

Are the following statements on naive bayes correct?



Bernoulli Naïve Bayes is a discriminative model in supervised learning

0

It models $P(x|y)$ as a product of independent Bernoulli distributions.

0

In classification, if we model $P(Y)$ using a multinomial distribution, then the MLE is always the empirical frequency, regardless of whether the underlying model is Naïve Bayes or not.

0

Are the following statements on naive Bayes correct?



Optimizing the parameters of Bernoulli Naïve Bayes is to find the set of parameters that fit the data the best, and we can achieve this by finding the set of parameters that maximize the log-likelihood of the data

0

Bernoulli Naïve Bayes cannot be solved in closed-form, so we need to run gradient descent.

0

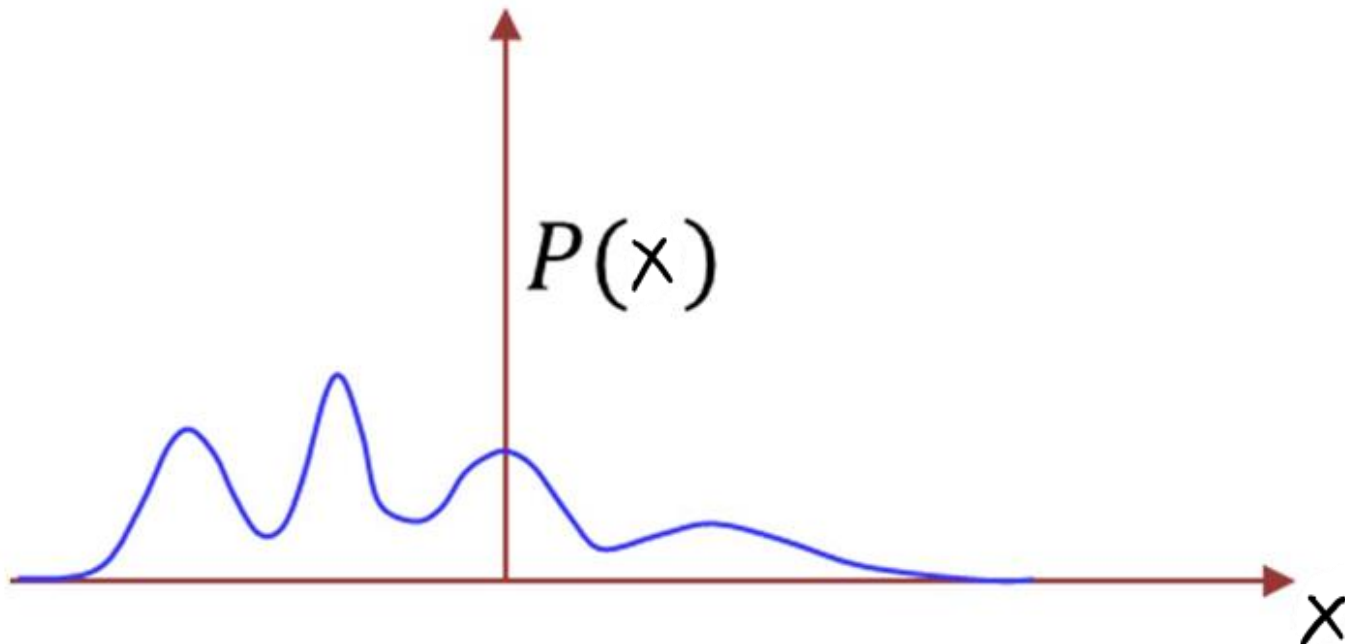
Naïve Bayes was the state-of-art for a very long time. It scales well to large datasets and is always a good baseline to try for text classification.

0

Gaussian Discriminant Analysis

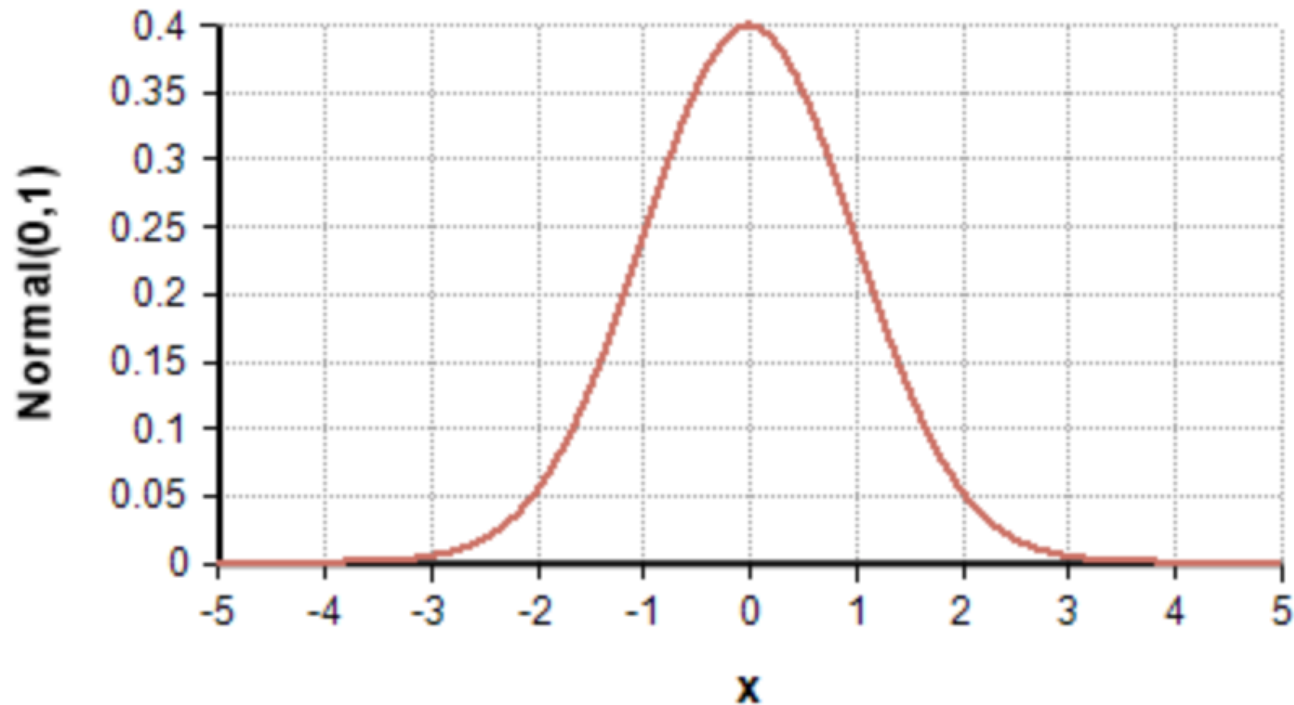
Gaussian Mixture Models (GMMs)

- Bernoulli Naïve Bayes is a particular way of modeling $P(\mathbf{x}|y)$ for text classification
- If the features are continuous, we want to use a better way to model continuous random variables
- For example, the distribution of X is of the following shape



Gaussian Mixture Models

- Standard Gaussian Distribution $N(u, \sigma^2)$



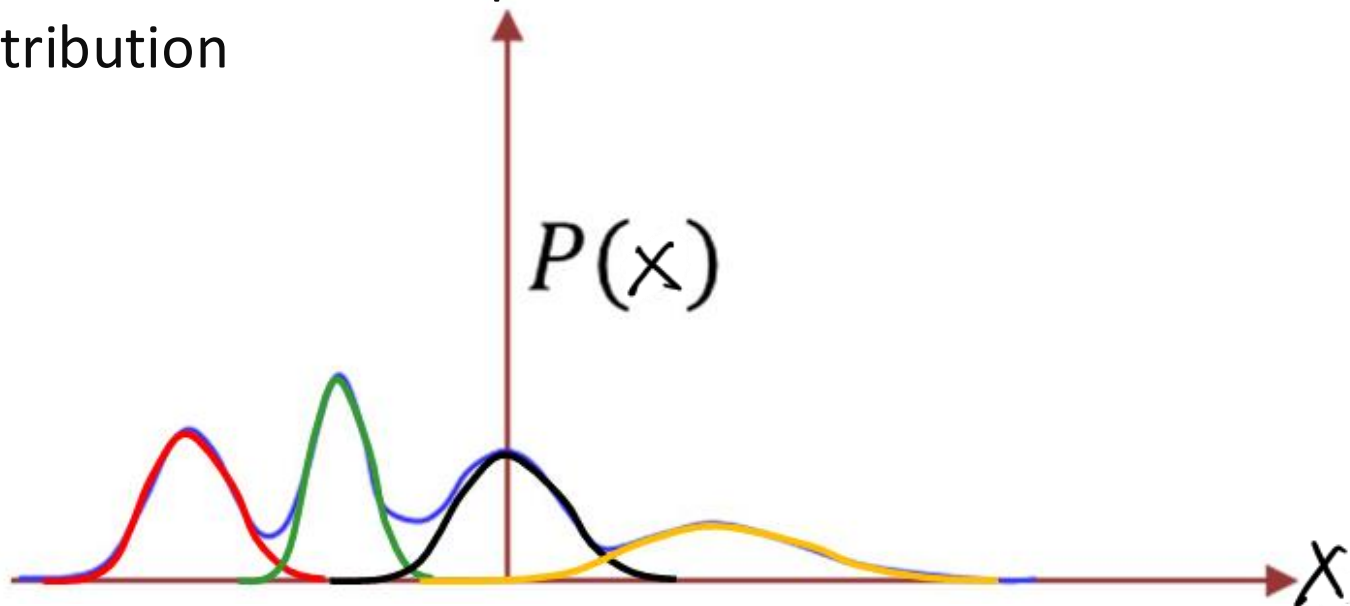
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Gaussian Mixture Models (GMMs)

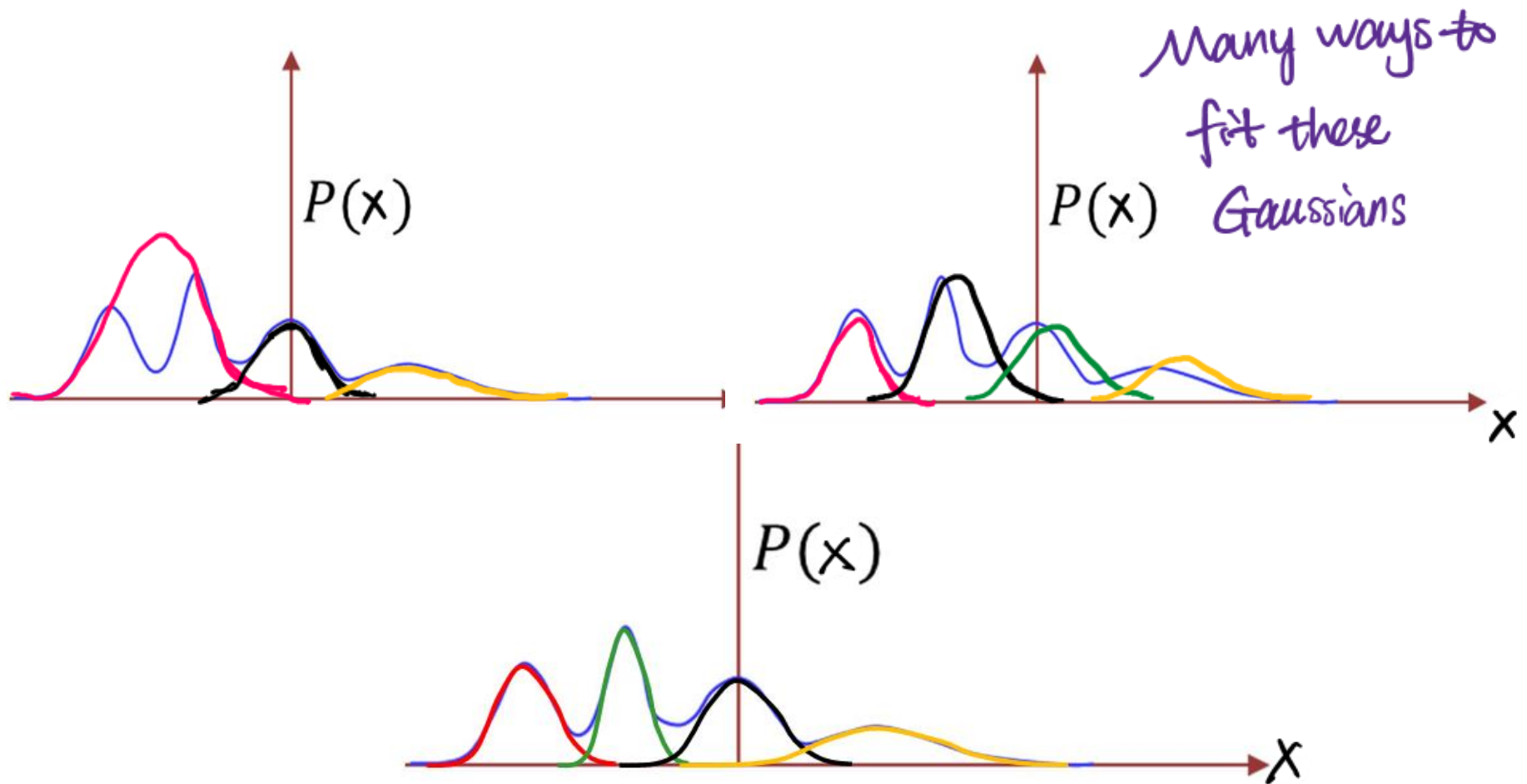
- We model it as a mixture of Gaussians (GMMs)
- Weighted sum of Gaussians

$$P(x) = \sum_k P(k) N(x; \mu_k, \sigma_k^2)$$

- The weights $p(k)$ sum to 1, each representing the likelihood that a data point is from each Gaussian distribution



Gaussian Mixture Models (GMMs)



Goal: given the samples from the original distribution, how to determine the parameters of GMM to best fit them?

GMM as Generative Classification Model

Mixture of Gaussians (GMMs): With $p(k)$'s sum to 1

$$P(x) = \sum_k P(k)N(x; \mu_k, \sigma_k^2)$$

- The number of mixtures corresponds to the number of label classes:
- $P(k) \rightarrow P(Y = k)$
- Each mixture corresponds to the conditional distribution of feature given label
- $N(x; \mu_k, \sigma_k^2) \rightarrow P(x|Y = k)$

$$P(x) = \sum_k P(Y = k)P(x|Y = k)$$